Evaluación de un modelo de embeddings: relevancia semántica vs. coincidencia de palabras clave

Modelo evaluado: **gemini-embedding-001** (3072 dimensiones) | Concepto: **Despliegue de microservicios** | Fecha de ejecución: 2026-11-09

1. Objetivo y metodología

Se evalúa si un modelo de embeddings distingue *significado* de *vocabulario compartido*. Se definen 5 oraciones que describen el mismo concepto técnico con vocabulario distinto (ninguna contiene las palabras "despliegue", "microservicios" ni "servicio") y 2 oraciones trampa que sí comparten palabras clave pero no significan lo mismo. Se mide la similitud coseno de cada oración contra el concepto y se la compara contra un baseline puramente léxico (TF-IDF). Toda la comparación se ejecutó 3 veces para comprobar que el resultado es estable.

2. Conceptos clave

Embedding: vector numérico denso que representa el significado de un texto; textos con significado parecido quedan cerca en el espacio vectorial, aunque no compartan palabras.

Similitud coseno: mide el ángulo entre dos vectores: cos(a,b) = (a·b) / (■a■·■b■). Vale 1 si apuntan en la misma dirección, 0 si son ortogonales y -1 si son opuestos. Ignora la magnitud, por eso es la métrica estándar para comparar embeddings de textos de distinta longitud.

Coincidencia léxica (TF-IDF): representa el texto por las palabras que contiene. Si dos textos no comparten palabras, su similitud es 0 aunque hablen de lo mismo; si las comparten, es alta aunque el significado sea opuesto. Es el comportamiento que se quiere contrastar.

3. Conjunto de oraciones

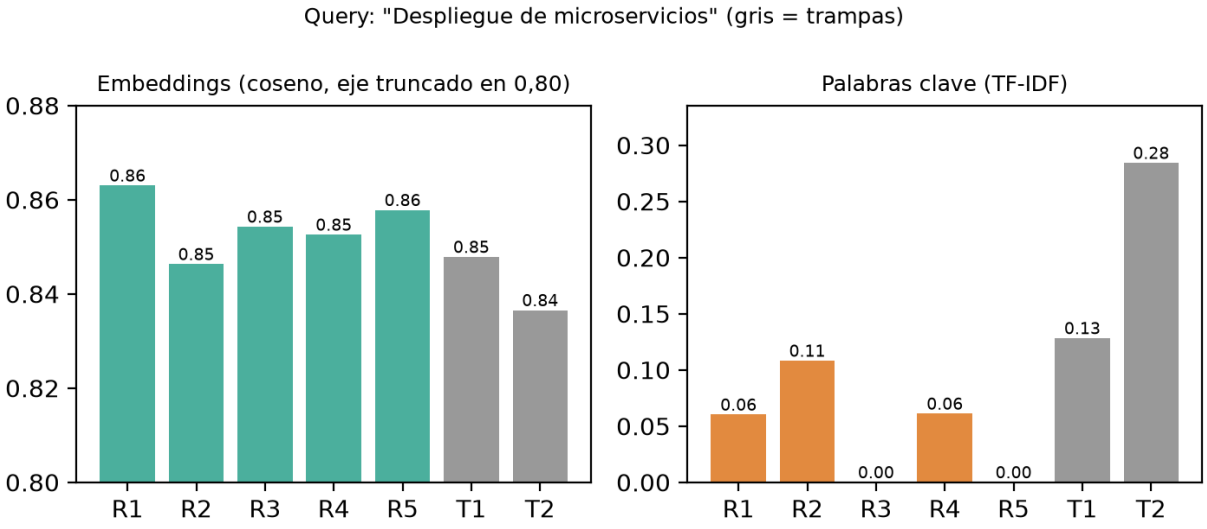
ID	Tipo	Oración
R1	Relevante (vocabulario distinto)	Cada componente pequeño de la aplicación se empaqueta en un contenedor Docker y se orquesta con Kubernetes para salir a producción.
R2	Relevante (vocabulario distinto)	Publicamos los módulos independientes del sistema en el clúster mediante un pipeline de CI/CD que construye imágenes y las lanza sin tiempo de inactividad.
R3	Relevante (vocabulario distinto)	El equipo libera en producción cada API autónoma por separado, usando contenedores y rollouts graduales.
R4	Relevante (vocabulario distinto)	Llevar a producción una arquitectura distribuida en piezas desacopladas exige automatizar la entrega, el escalado y la reversión de versiones.
R5	Relevante (vocabulario distinto)	Subimos al clúster en la nube cada pieza liviana y escalable por separado, con entregas automatizadas y sin detener el sistema.
T1	Trampa: irrelevante (homónimos)	El servicio de micro-limpieza es excelente: despliegan un equipo de personal en cada oficina todas las mañanas.
T2	Trampa: significado opuesto	Nuestro monolito se despliega como un único bloque; los microservicios no aportan nada y rechazamos su despliegue.

T1 comparte "servicio", "micro" y "despliegan" pero habla de limpieza de oficinas. T2 comparte "microservicios" y "despliegue" pero rechaza la idea (postura opuesta).

4. Resultados

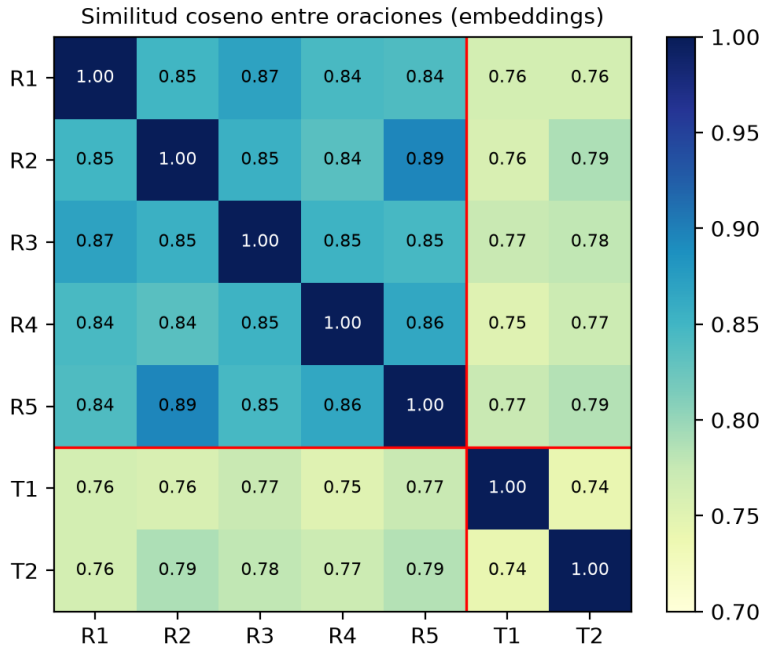
4.1 Query corta contra cada oración

ID	Coseno (embeddings)	TF-IDF (palabras clave)	Posición semántica
R1	0.8630	0.0602	1
R5	0.8578	0.0000	2
R3	0.8543	0.0000	3
R4	0.8526	0.0610	4
T1	0.8479	0.1282	5
R2	0.8463	0.1081	6
T2	0.8365	0.2841	7



Lectura. El baseline léxico asigna 0,00 a dos oraciones relevantes (R3, R5) y su puntaje más alto a la trampa T2 (0.28): premia justamente las palabras repetidas. Los embeddings asignan puntajes altos a las 5 oraciones relevantes y colocan a T2 en último lugar. **Limitación observada:** con una query de solo 3 palabras los scores quedan comprimidos en un rango muy estrecho (0.837 a 0.863) y R2 (0.8463) queda por debajo de la trampa homónima T1 (0.8479), una diferencia de 0.0015. La separación existe pero es frágil para queries tan cortas; el margen no alcanza para fijar un umbral de corte fiable.

4.2 Oración contra oración (más contexto)



Similitud promedio entre oraciones relevantes: **0.855**. Similitud promedio entre una relevante y una trampa: **0.770**. La brecha (0.085) es mucho mayor que la diferencia observada con la query corta.

4.3 Leave-one-out: cada oración relevante como query

Query	Ranking de las demás oraciones (mayor a menor)	Trampas al final
R1	R3 > R2 > R4 > R5 > T1 > T2	Sí
R2	R5 > R3 > R1 > R4 > T2 > T1	Sí
R3	R1 > R2 > R4 > R5 > T2 > T1	Sí
R4	R5 > R3 > R1 > R2 > T2 > T1	Sí
R5	R2 > R4 > R3 > R1 > T2 > T1	Sí

En los 5 casos las dos trampas quedan en las últimas posiciones: cuando la query tiene contexto suficiente, el modelo discrimina correctamente, incluso la trampa T2 que comparte las palabras exactas.

4.4 Verificación repetida

Corrida	Ranking semántico (query corta)	Diferencia máx. vs corrida 1
1	R1 > R5 > R3 > R4 > T1 > R2 > T2	0.00e+00
2	R1 > R5 > R3 > R4 > T1 > R2 > T2	0.00e+00
3	R1 > R5 > R3 > R4 > T1 > R2 > T2	0.00e+00

Resumen de chequeos automáticos: ranking idéntico entre corridas = **PASS**; leave-one-out con trampas al final = **PASS**; baseline léxico elige una trampa como primera = **PASS**; query corta: todas las relevantes sobre T2 = **PASS**; query corta: todas las relevantes sobre T1 = **FAIL** (este último falla y se reporta tal cual).

5. Similitud coseno con scikit-learn

sklearn.metrics.pairwise.cosine_similarity(X, Y) recibe arreglos 2D de forma (n_muestras, n_dimensiones) y devuelve una matriz con el coseno entre cada fila de X y cada fila de Y. Para dos vectores sueltos hay que darles forma (1, n) con reshape(1, -1) o listas anidadas. El código siguiente se ejecutó y produce los valores indicados en los comentarios.

```
"""Minimal cosine similarity example with scikit-learn (deterministic, no API)."""

import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# Two numeric vectors (e.g. embeddings with 4 dimensions for readability).
a = np.array([[0.2, 0.8, 0.1, 0.5]])
b = np.array([[0.3, 0.7, 0.0, 0.6]])

# scikit-learn expects 2D arrays: (n_samples, n_features).
sklearn_score = cosine_similarity(a, b)[0][0]

# Same value computed by hand: dot(a, b) / (||a|| * ||b||).
manual_score = float(np.dot(a[0], b[0]) / (np.linalg.norm(a) * np.linalg.norm(b)))

print(f"sklearn cosine similarity: {sklearn_score:.6f}")
print(f"manual cosine similarity:  {manual_score:.6f}")
assert np.isclose(sklearn_score, manual_score)

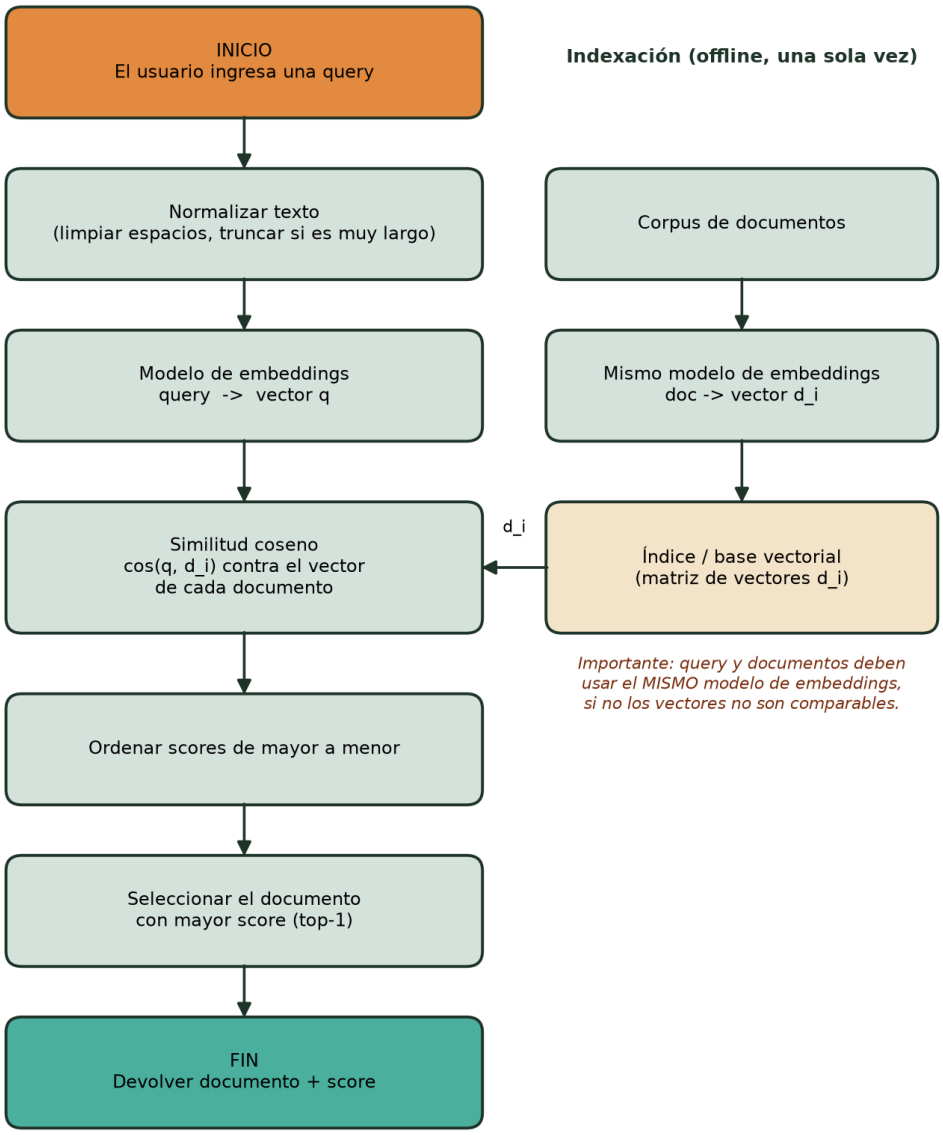
# Edge cases that explain how to read the score.
print("identical direction:", cosine_similarity([[1, 2]], [[2, 4]])[0][0])   # 1.0
print("orthogonal:         ", cosine_similarity([[1, 0]], [[0, 1]])[0][0])   # 0.0
print("opposite direction: ", cosine_similarity([[1, 2]], [[-1, -2]])[0][0]) # -1.0
```

Cálculo aplicado al pipeline de este ejercicio (de similarity.py):

```
def cosine_between(query, documents):
    # query: (dims,)  documents: (n_docs, dims)  ->  scores: (n_docs,)
    return cosine_similarity(query.reshape(1, -1), documents)[0]

vectors = embed_texts([CONCEPT] + texts)      # Gemini, task_type=SEMANTIC_SIMILARITY
semantic = cosine_between(vectors[0], vectors[1:])
lexical = keyword_similarity(CONCEPT, texts)   # TF-IDF baseline
```

6. Diagrama de flujo: búsqueda semántica



El flujo de la izquierda se ejecuta en cada consulta; el de la derecha se ejecuta una vez (o cada vez que cambia el corpus) y deja los vectores listos para comparar. Implementación: semantic_search.py.

7. Conclusiones

1) El modelo agrupa correctamente oraciones con vocabulario totalmente distinto, y el baseline léxico demuestra por contraste que la palabra clave por sí sola engaña (puntúa 0 a paráfrasis válidas y más alto a la trampa de postura opuesta).
2) Con una query corta de tres palabras, la ventaja de los embeddings existe pero es pequeña y una trampa homónima (T1) supera a una paráfrasis (R2) por 0,0015.
3) Con oraciones completas como query, la separación es clara y estable.
4) Recomendación de arquitectura: no depender de un umbral de coseno fijo con queries cortas; enriquecer la query, usar un reranker o combinar con búsqueda léxica (híbrida), y validar siempre con un conjunto de pruebas con trampas como este.